Python Fundamentals II

Data Analytics Bootcamp

Today

1. **Conditional Statements**
    a. Basic Statements
    b. Advanced Statements
    c. **Exercise**: Python Conditionals
2. **Functions**
    a. Basic Functions
    b. What Are Parameters & Arguments?
    c. Parameter & Argument Relationship
    d. **Exercise**: Python Functions

Topic

Conditional Statements

Conditional Statements

Conditional statements allow your program to make **decisions**.

- Run different instructions based on **yes/no** questions
- Create different **paths** your program can follow

"Yes/no" equates to "true/false".

This is boolean logic.

Conditional Statements

Basic Statements

1. `if` Statements
2. `if else` Statements
3. `if-elif-else` Statements

Advanced Statements

1. Logical Operators
2. Nested Statements
3. Ternary Statements

`if` Statements

The *basic* conditional statement.

```
if condition:
    do_something()
```

```
age = 18
if age >= 18:
    print('You are an adult')
```

`if else` Statements

Adds an alternative path.

```
if condition:
    do_something()
else:
    do_something_else()
```

```
age = 16
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

`if-elif-else` Statements

Handling multiple conditions with branching paths.

```
if condition1:
    do_something()
elif condition2:
        do_something_else()
else:
    default_action()
```

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

Logical Operators

Expand the possible conditions.

1. `and`
   - **Both** conditions must be **true**.
2. `or`
   - **One** condition must be **true**.
3. `not`
   - **Flips** the boolean.

```python
if age >= 18 and has_id:
    print("Can enter")

if is_member or is_staff:
    print("Access granted")

if not is_banned:
    print("Welcome")
```

Nested Statements

Conditions within conditions.

"Flat is better than nested."

Nesting statements does not align with the *Zen of Python.*

```
if outer_condition:
    if inner_condition:
        do_something()
    else:
        do_something_else()
```

```
is_logged_in = True
is_admin = True

if is_logged_in:
    if is_admin:
        print("Admin dashboard")
    else:
        print("User dashboard")
else:
    print("Please log in")
```

Ternary Statements

A way to write more with less.

```
value_if_true if condition else value_if_false
```

```
age = 20
status = "adult" if age >= 18 else "minor"
```

"Readability counts."

Ternary statements don't contradict the *Zen of Python*, but too many will.

Conditional Best Practices

Keep conditions **simple** and **readable**

Use **meaningful** variable names

Consider the **order** of conditions.

Exercise

Python Conditionals

Topic

Functions

What Are Functions?

Functions allow you to **organize** and **reuse** code.

- **Group** related operations together
- **Communicate** purpose with meaningful naming
- Write code **once**, execute it whenever needed
- Allow different inputs for **flexible** behavior

Organize

Group Together & Communicate Purpose

- Collects all statistical calculations together.
- Name “calculate_statistics” makes clear its purpose.

```
def calculate_statistics(numbers):
    mean = sum(numbers) / len(numbers)
    median = sorted(numbers)[len(numbers) // 2]
    return mean, median
```

Reuse

Write Once & Flexible Behavior

- Calls the function to reuse the code.
- Allows the code to be used for calculating both test scores and temperatures.

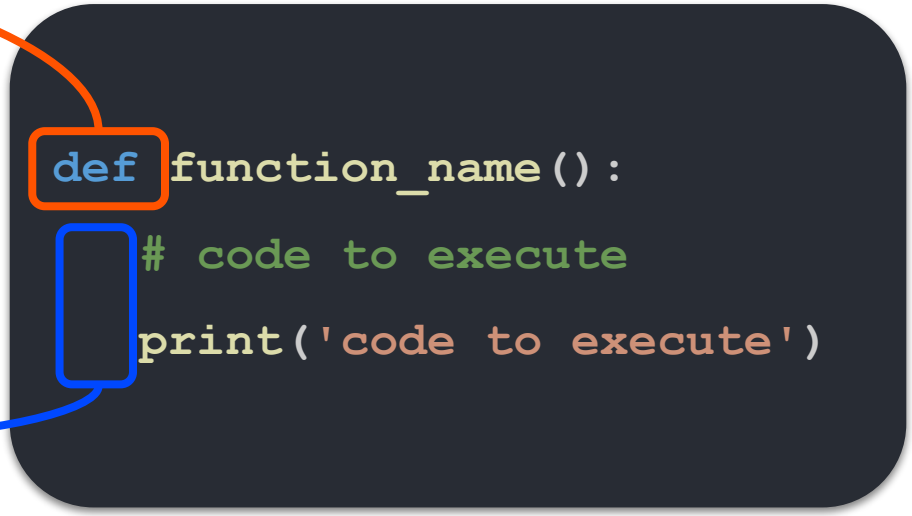
```
test_scores = [85, 92, 78, 90, 88]
mean_score, median_score = calculate_statistics(test_scores)
print(f"Test scores - Mean: {mean_score}, Median: {median_score}")

temperatures = [23.5, 24.1, 22.8, 25.0, 23.2, 24.5]
mean_temp, median_temp = calculate_statistics(temperatures)
print(f"Temperatures - Mean: {mean_temp}, Median: {median_temp}")
```

Basic Function Syntax

The def keyword tells Python you're creating a function.

The function runs the indented code.



Basic Function Use

The function `greet` will print its message to the console when called.

Using `greet()` is what calls the function to run.

```
def greet():
    print("Hello, world!")

greet()  # Outputs: Hello, world!
```

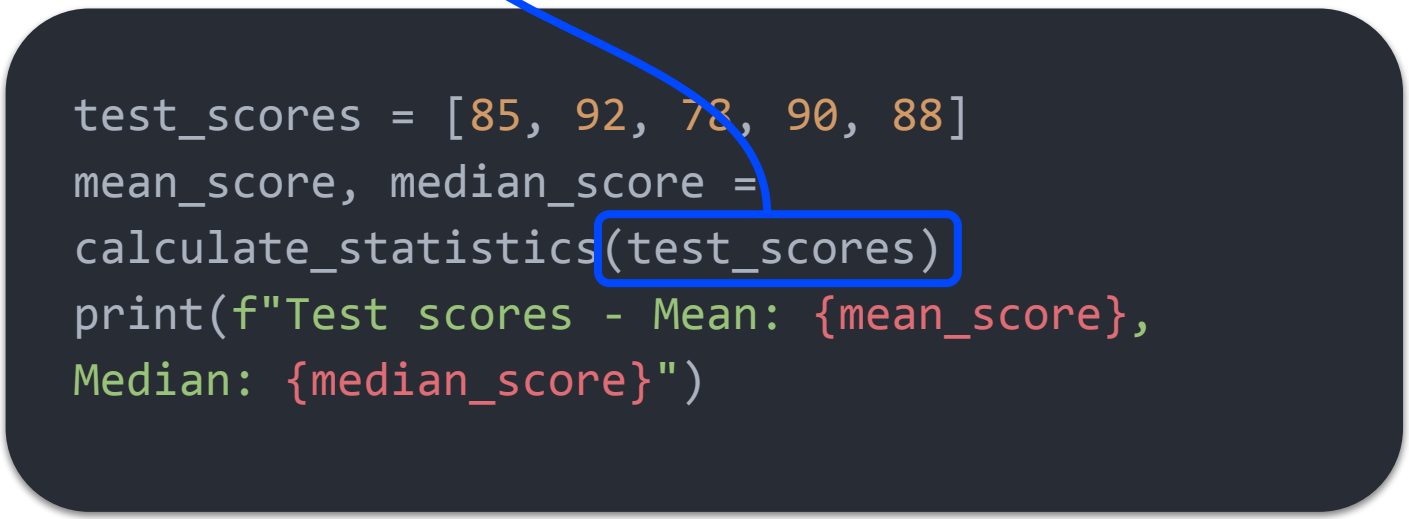
What Are Parameters?

Parameters are **variables** placed within the parentheses that act as a **placeholder** for your inputs.

```
def calculate_statistics(numbers):
    mean = sum(numbers) / len(numbers)
    median = sorted(numbers)[len(numbers) // 2]
    return mean, median
```

What Are Arguments?

Arguments are the **input** passed to a function that become the parameter's value.



Passing in **`test_scores`** means it is now the value of **`numbers`** when the function executes.

```
def calculate_statistics(numbers):
    mean = sum(numbers) / len(numbers)
    median = sorted(numbers)[len(numbers) // 2]
    return mean, median

test_scores = [85, 92, 78, 90, 88]
mean_score, median_score =
calculate_statistics(test_scores)
```

```
def calculate_statistics([85, 92, 78, 90, 88]):
    mean = sum([85, 92, 78, 90, 88]) / len([85, 92, 78, 90, 88])
    median = sorted([85, 92, 78, 90, 88])[len([85, 92, 78, 90, 88]) // 2]
    return mean, median
```

Parameters & Arguments Relationship

Exercise

Python Functions